

PUBLIC KEY CRYPTOGRAPHY BASICS

[Kaynak](#)

İÇERİK

1. Kriptografiye Giriş: Gerçek Hayattan Siber Dünyaya Güvenlik
2. Asimetrik Şifrelemenin En Yaygın Kullanımı: Anahtar Değişimi (Key Exchange)
3. RSA: Asimetrik Şifrelemenin Matematiksel Temeli
4. Diffie-Hellman Anahtar Değişimi (Key Exchange)
5. SSH ve Anahtar Tabanlı Kimlik Doğrulama
6. Dijital İmzalar ve Sertifikalar
7. PGP ve GPG: E-posta ve Dosya Güvenliğinin Standartları
8. Sonuç: Kriptografi ve Kriptanaliz Dengesi

Kriptografiye Giriş: Gerçek Hayattan Siber Dünyaya Güvenlik

Kriptografiyi sadece karmaşık matematiksel formüller olarak değil, güvenli bir iletişimin temel taşları olarak görmeliyiz. Günlük hayattaki sıradan bir kahve toplantısı ile dijital dünyadaki veri transferi arasında güvenlik prensipleri açısından hiçbir fark yoktur.

Gerçek Hayat Senaryosu: Kahve Sohbeti Üzerinden Analiz

Bir iş ortağınızla kafede buluşup gizli bir proje konuştuğunuzu hayal edin. Bu basit etkileşim aslında dört ana güvenlik sütununa dayanır:

1. **Authentication (Kimlik Doğrulama):** Karşınızdaki kişiyi görüyorsunuz ve sesini tanıyorsunuz. Onun, konuştuğunuzu iddia ettiği kişi olduğundan eminsiniz. Yani **kimliğini doğruluyorsunuz**.
2. **Authenticity (Özgünlük/Kaynağın Doğruluğu):** Kelimelerin ve cümlelerin doğrudan ortağınızın ağzından çıktığını teyit edebiliyorsunuz. Mesajın gerçekten iddia edilen kaynaktan geldiğini biliyorsunuz.
3. **Integrity (Bütünlük):** Masanın bir ucundan diğerine giden sözcüklerin havada değişmediğinden eminsiniz. Ortağınızın "Evet" dediği şeyi kulağınızın "Hayır" olarak duyması mümkün değil; **veri bozulmamış veya değiştirilmemiştir**.
4. **Confidentiality (Gizlilik):** Diğer müşterilerden uzak bir masaya oturup alçak sesle konuşuyorsunuz. Amacınız, veriye (sohbete) sadece **yetkili kişilerin** erişmesini sağlamaktır.

Siber Dünyada Gereksinimler

Dijital ortamda iş ortağınıza bir mesaj gönderdiğinizde, karşınızda fiziksel bir varlık yoktur. Bu durumda yukarıdaki dört unsuru sağlamak için kriptografik çözümlere ihtiyaç duyarız:

Güvenlik Unsuru	Siber Dünyadaki Anlamı	Kriptografik Çözüm Odağı
Authentication	Doğru kişiyle mi yazışıyorum yoksa bir taklitçiyle mi?	Dijital Sertifikalar, Biometrikler
Authenticity	Gelen bilgi gerçekten iddia edilen kaynaktan mı geliyor?	Dijital İmzalar (Asimetrik)
Integrity	Mesaj ağ üzerinden geçerken bir saldırgan tarafından değiştirildi mi?	Hash Fonksiyonları, MAC
Confidentiality	Mesajı iletirken biri hattı dinliyor mu (eavesdropping)?	Şifreleme (Simetrik/Asimetrik)

Kriptografinin Rol Dağılımı

Kriptografi bu gereksinimleri karşılamak için iki ana silah kullanır:

- Simetrik Şifreleme (Private Key):** Temel amacı **Gizliliği (Confidentiality)** korumaktır. Veriyi hızlıca şifreler, ancak kimlik doğrulama veya bütünlük konusunda (tek başına) zayıf kalabilir.
- Asimetrik Şifreleme (Public Key):** Gizliliğin yanı sıra; **Authentication**, **Authenticity** ve **Integrity** sağlanmasında kritik bir rol oynar. Dijital imzaların ve güvenli anahtar değişimlerinin temelidir.

Kritik Not: Modern siber güvenlikte "güvenli iletişim" demek, bu dört unsurun aynı anda sağlanması demektir. Sadece şifreleme yapmak (gizlilik), karşıdaki kişinin kimliğinden emin olmadığınız sürece (authentication) yeterli değildir.

Asimetrik Şifrelemenin En Yaygın Kullanımı: Anahtar Değişimi (Key Exchange)

Asimetrik şifreleme, simetrik şifrelemeye göre matematiksel olarak çok daha ağır ve yavaştır. Bu yüzden gigabaytlarca veriyi RSA ile şifrelemek verimli değildir. Modern sistemler bu sorunu hibrit bir yaklaşımla çözer: **Asimetrik şifreleme, simetrik şifreleme anahtarları üzerinde "tokalaşmak" (negotiate) için kullanılır.**

Büyük Soru: Anahtarı Nasıl Güvenli Paylaşırız?

Eğer simetrik şifreleme kullanacaksak, her iki tarafın da aynı gizli anahtara sahip olması gerekir. Ancak bu anahtarı internet üzerinden birbirimize gönderirsek, hattı dinleyen bir saldırgan (**snooping**) bu anahtarı ele geçirip tüm iletişimi çözebilir. İşte asimetrik şifreleme burada devreye girer.

Hayatın İçinden Bir Analoji: Kilitli Kutu Metodu

İş ortağınızla gizli bir kod (simetrik anahtar) üzerinde anlaşmak istediğinizi hayal edin:

- Kilit İstemek:** Arkadaşınızdan size bir **asılı kilit** göndermesini istersiniz. Bu kilidin anahtarı sadece arkadaşınızdadır.
- Kilitlemek:** Gizli kodu yazıp kırılmaz bir kutuya koyarsınız ve arkadaşınızın gönderdiği o kilit ile kutuyu kilitlersiniz.
- Göndermek:** Kilitli kutuyu arkadaşınıza gönderirsiniz. Yolda bu kutuyu gören herkes içinde bir şeyler olduğunu bilir ama kilidi açacak anahtar sadece arkadaşınızda olduğu için kutuyu açamazlar.
- Açmak:** Arkadaşınız kutuyu kendi anahtarıyla açar, gizli kodu öğrenir.
- İletişim:** Artık ikiniz de gizli kodu bildiğinize göre, kutularla uğraşmadan bu kodla hızlıca haberleşebilirsiniz.

Kriptografik Karşılıklar

Bu analogideki unsurların teknik karşılıkları şöyledir:

Analoji	Kriptografik Sistem	Karşılığı
Gizli Kod	Symmetric Key	Veriyi şifreleyecek asıl hızlı anahtar.
Kilit	Public Key	Herkese açık, sadece kilitlemeye (şifrelemeye) yarar.
Kilidin Anahtarı	Private Key	Sadece sahibinde olan, kilidi açmaya (şifre çözmeye) yarar.

Neden Bu Yöntemi Seçiyoruz?

- Performans:** Asimetrik kriptografiyi sadece **bir kez** (bağlantı kurulurken) kullanırsınız. Anahtar üzerinde anlaşıldıktan sonra iletişimin geri kalanı çok daha hızlı olan simetrik şifreleme (örneğin AES) ile devam eder. Bu sayede hız kaybı yaşanmaz.
- Güvenlik:** Gizli anahtar internet kablolarından asla "açık" (cleartext) bir şekilde geçmez. Sadece alıcının çözebileceği bir formda paketlenmiş olur.

Gerçek Dünyadaki Ekstra Katmanlar

Yukarıdaki senaryo gizliliği sağlar, ancak bir sorun daha vardır: **"O kilidi bana gerçekten arkadaşım mı gönderdi, yoksa araya giren bir saldırgan kendi kilidini mi yolladı?"**

Bu problemi (Authentication) çözmek için asimetrik şifrelemenin diğer iki büyük silahı devreye girer:

- Digital Signatures (Dijital İmzalar):** Mesajın kimden geldiğini kanıtlar.

- **Certificates (Sertifikalar):** Public key'in (kilidin) gerçekten o kişiye ait olduğunu onaylayan dijital kimlik kartlarıdır.

RSA: Asimetrik Şifrelemenin Matematiksel Temeli

RSA (Rivest-Shamir-Adleman), güvensiz kanallar (saldırganların dinleme yapabildiği ağlar) üzerinden güvenli veri iletimini sağlayan en temel ve yaygın asimetrik şifreleme algoritmasıdır.

RSA Güvenliğinin Arkasındaki Matematik: Çarpanlara Ayırma Problemi

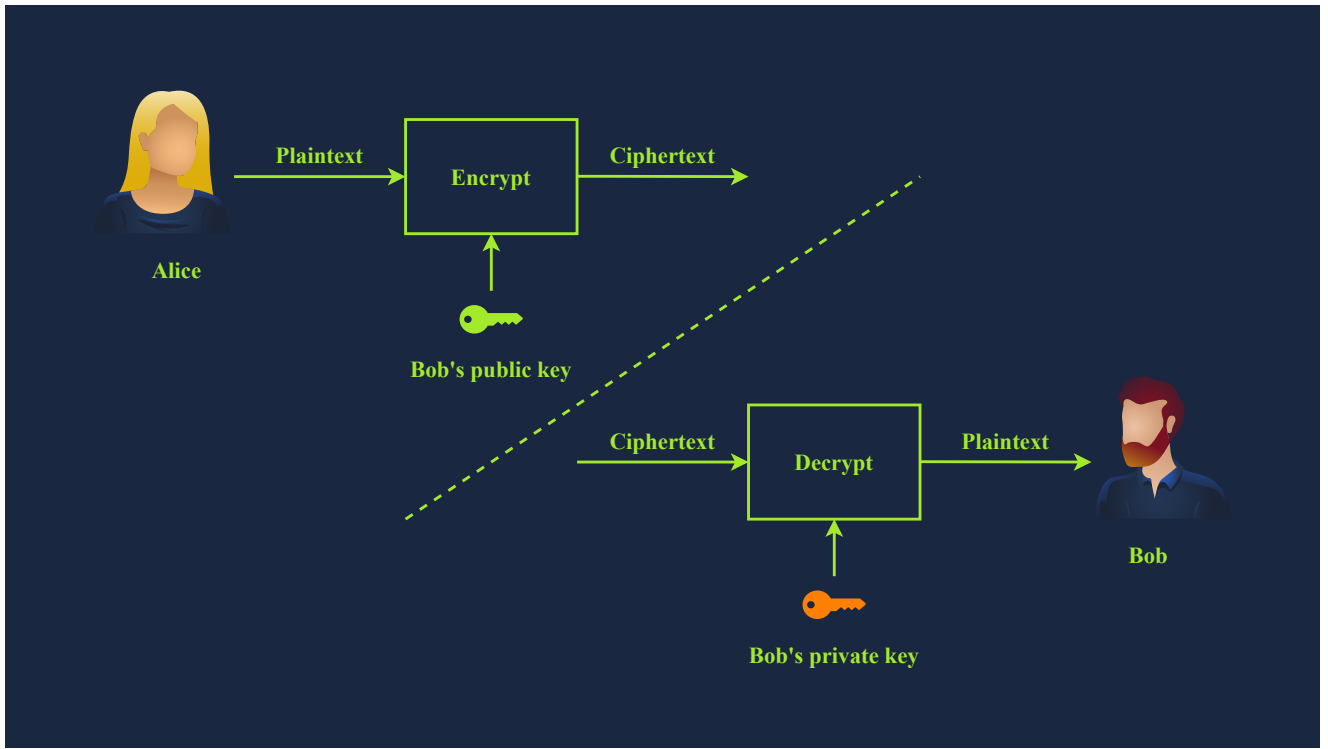
RSA'nın güvenliği, **Integer Factorization** (Tam Sayıları Çarpanlara Ayırma) probleminin zorluğuna dayanır.

- **Kolay İşlem:** İki çok büyük asal sayıyı birbiriyle çarpmak bilgisayarlar için çok kolay ve hızlı bir işlemdir.
 - *Örnek:* $113 \times 127 = 14351$ (Elle bile kolay).
- **Zor İşlem:** Çok büyük bir sayının (örneğin 600 haneli) hangi iki asal sayının çarpımı olduğunu bulmak (çarpanlara ayırmak) mevcut bilgisayar gücüyle pratik olarak imkansızdır (**computationally infeasible**).

Gerçek dünyada RSA, her biri yaklaşık **300 hane** olan iki devasa asal sayıyı kullanır. Çarpımları olan **n** değeri kamuya açık olsa da, onu oluşturan **p** ve **q** asal sayılarını bulmak milyonlarca yıl sürebilir.

RSA Algoritmasının Çalışma Mantığı (Adım Adım)

Aşağıdaki basitleştirilmiş nümerik örnek üzerinden RSA'nın nasıl "anahtar ürettiğini" ve veriyi nasıl işlediğini görelim:



1. Anahtar Oluşturma (Bob'un Tarafı)

- **Asal Sayı Seçimi:** Bob iki asal sayı seçer: $p = 157$ ve $q = 17$.
- **n Hesaplama:** $n = p \times q = 2669$.
- **Totient Fonksiyonu ($\phi(n)$):** $\phi(n) = (p - 1) \times (q - 1) = 156 \times 16 = 2512$.
- **e Seçimi (Public Exponent):** $\phi(n)$ ile arasında asal olan bir sayı seçilir. Örneğin: $e = 163$.
- **d Hesaplama (Private Exponent):** $e \times d \equiv 1 \pmod{\phi(n)}$ denklemini sağlayan d bulunur. Örneğimizde $d = 1541$.

Public Key (Açık Anahtar): (n, e) $(2669, 163)$ — Bunu herkes bilebilir.

Private Key (Gizli Anahtar): (n, d) $(2669, 1541)$ — Sadece Bob bilir.

2. Şifreleme (Alice'in Tarafı)

Alice, Bob'a $x = 13$ değerini göndermek istiyor:

$$y = x^e \pmod{n} = 13^{163} \pmod{2669} = 16341$$

Alice ağ üzerinden **16341** değerini gönderir.

3. Şifre Çözme (Bob'un Tarafı)

Bob gelen şifreli metni (y) kendi gizli anahtarı (d) ile çözer:

$$x = y^d \pmod{n} = 16341^{1541} \pmod{2669} = 13$$

Bob, orijinal mesaj olan **13** değerine ulaşır.

CTF'lerde RSA ve Kritik Değişkenler

CTF (Capture The Flag) yarışmalarında kriptografi soruları genellikle RSA matematiğini manipüle etmenizi bekler. Karşılaşacağınız değişkenler şunlardır:

- **p, q:** Gizli tutulması gereken devasa asal sayılar.
- **n:** Modulus ($p * q$). Hem açık hem kapalı anahtarda bulunur.
- **e:** Public exponent. Genellikle 65537 olarak sabit seçilir.
- **d:** Private exponent. Şifreyi çözen asıl gizli değerdir.
- **m (veya x):** Message (Plaintext).
- **c (veya y):** Ciphertext (Şifreli mesaj).

CTF İçin Araç Tavsiyeleri

Eğer bir CTF'de size bu değerlerden birkaçı verildiyse ve mesajı çözmeniz isteniyorsa şu araçlar hayat kurtarır:

1. **RsaCtfTool:** En popüler araçtır. Bilinen pek çok RSA zafiyetini otomatik olarak tarar ve şifreyi kırmaya çalışır.
2. **rsatool:** Özel anahtar (private key) oluşturmak veya değişkenler arası hesaplama yapmak için kullanışlıdır.
3. **Factordb.com:** Eğer size verilen **n** değeri çok büyük değilse, bu site üzerindeki veritabanında daha önce çözülmüş asal çarpanlarını (**p** ve **q**) bulabilirsiniz.

Diffie-Hellman Anahtar Değişimi (Key Exchange)

Simetrik şifrelemenin en büyük sorunu, tarafların aynı anahtara nasıl sahip olacağıdır. Eğer anahtarı internet üzerinden açıkça gönderirseniz, bir saldırgan onu çalabilir. **Diffie-Hellman (DH)**, iki tarafın güvensiz bir kanal üzerinden (herkesin dinleyebildiği bir hat) birbirlerine gizli anahtarı göndermeden, ortak bir gizli anahtar üzerinde anlaşmalarını sağlayan devrim niteliğinde bir protokoldür.

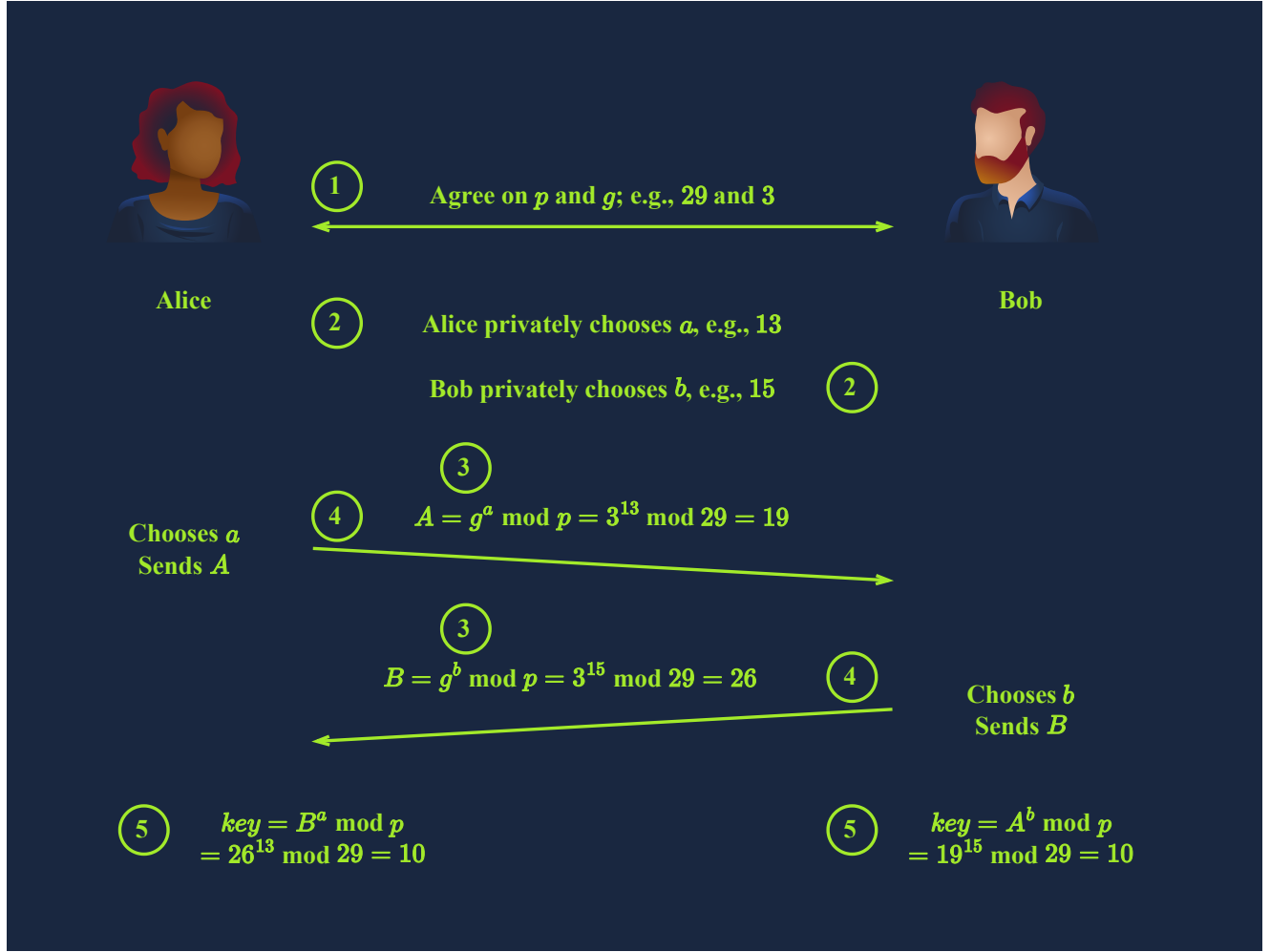
1. Mantıksal Analoji (Renk Karıştırma Metodu)

Matematiğe girmeden önce, DH'nin çalışma mantığını renkler üzerinden düşünelim:

1. **Ortak Malzeme:** Alice ve Bob herkesin görebileceği bir "ortak renk" (örneğin Sarı) seçer.
2. **Gizli Renkler:** Alice kendine gizli bir renk seçer (Kırmızı), Bob da kendine (Mavi). Bu renkleri kimseye söylemezler.
3. **Karıştırma ve Paylaşma:** Alice kendi gizli rengini sarı ile karıştırır (Turuncu elde eder) ve Bob'a gönderir. Bob da kendi gizlisini sarı ile karıştırır (Yeşil elde eder) ve Alice'e gönderir.
4. **Sonuç:** Alice, Bob'dan gelen karışımın içine kendi gizli rengini ekler. Bob da Alice'ten gelen karışımın içine kendi gizli rengini ekler.
 - Sonuçta her iki tarafta da aynı son karışım (**Sarı + Kırmızı + Mavi**) oluşur.
 - Hattı dinleyen bir saldırgan sadece sarıyı, turuncuyu ve yeşili görür; ancak karışımları orijinal renklerine ayırmak imkansız olduğu için son gizli rengi asla elde edemez.

2. Matematiksel Süreç (Adım Adım)

Gerçek dünyada bu "karıştırma" işlemi **modüler üst alma** (modular exponentiation) ile yapılır.



Adım 1: Ortak Değişkenler

Alice ve Bob, herkesin görebileceği iki sayı üzerinde anlaşır:

- **p**: Çok büyük bir asal sayı (Örn: 29)
- **g**: Bir üreteç (generator), $0 < g < p$ (Örn: 3)

Adım 2: Özel Anahtarlar (Private Keys)

Her iki taraf da kimseyle paylaşmayacağı gizli bir tam sayı seçer:

- Alice: **a** = 13
- Bob: **b** = 15

Adım 3: Kamu Açık Anahtarların Hesaplanması (Public Keys)

Her taraf kendi gizli sayısını kullanarak bir "karışım" (açık anahtar) üretir:

- Alice: $A = g^a \mod p = 3^{13} \mod 29 = 19$
- Bob: $B = g^b \mod p = 3^{15} \mod 29 = 26$

Adım 4: Anahtar Değişimi

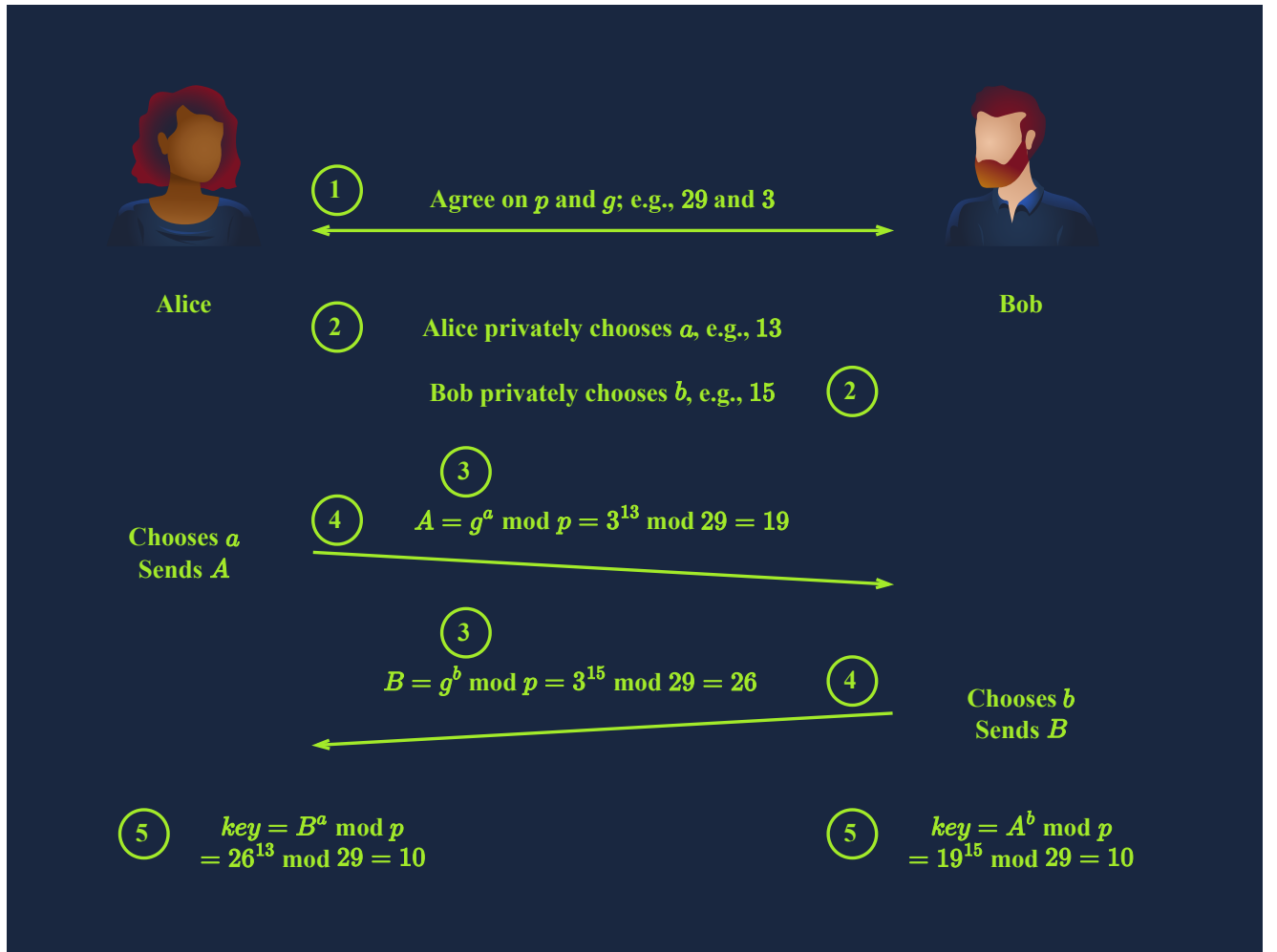
Alice Bob'a 19 değerini, Bob da Alice'e 26 değerini gönderir. Bir saldırgan bu iki sayıyı da görebilir ancak bunlar üzerinden gizli **a** ve **b** değerlerini hesaplayamaz (**Discrete Logarithm Problem**).

Adım 5: Ortak Gizli Anahtarın (Shared Secret) Hesaplanması

Her iki taraf, karşıdan gelen sayıyı kendi gizli anahtarıyla işler:

- Alice (Bob'un B'sini aldı): $= B^a \pmod{p} = 26^{13} \pmod{29} =$
- Bob (Alice'in A'sını aldı): $= A^b \pmod{p} = 19^{15} \pmod{29} =$

Sonuç: Her iki taraf da hiçbir anahtar transfer etmeden **10** sayısında buluştu! Artık bu sayıyı AES gibi simetrik algoritmalar için anahtar olarak kullanabilirler.



3. Güvenlik ve RSA ile İlişkisi

Diffie-Hellman harikadır ancak tek başına bir kusuru vardır: **Kimlik Doğrulama (Authentication) yapamaz.**

- **Zayıflık:** Alice, Bob ile konuştuğunu sanırken aslında araya giren bir saldırganla (Man-in-the-Middle) anahtar değişimi yapıyor olabilir.
- **Çözüm (Hibrit Yapı):** Diffie-Hellman genellikle **RSA** ile birlikte kullanılır.

- **Diffie-Hellman:** Ortak anahtarın güvenli bir şekilde oluşturulmasını (Key Agreement) sağlar.
- **RSA:** Dijital imzalar aracılığıyla "karşıdaki kişinin gerçekten Bob olduğunu" doğrulamak (Authentication) için kullanılır.

Günümüzde web trafiğinin çoğu (HTTPS/TLS), bu iki devin işbirliği sayesinde hem hızlı hem de güvenli bir şekilde gerçekleşir.

SSH ve Anahtar Tabanlı Kimlik Doğrulama

SSH (**Secure Shell**), güvensiz bir ağ üzerinden bir sunucuya güvenli bir şekilde erişmek için kullanılan protokoldür. Bu süreçte hem sunucunun hem de istemcinin (senin) kimliğinin doğrulanması gerekir.

1. Sunucunun Doğrulanması (Authenticating the Server)

Bir sunucuya ilk kez bağlandığında terminalde şöyle bir uyarı alırsın:

Bash

```
The authenticity of host '10.10.244.173' can't be established.  
ED25519 key fingerprint is  
SHA256:LLzhZc7YzRBDchm02qTX0qsLqeeiTCJg5ipOT0E/YM8.  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Bu Uyarı Ne Anlama Geliyor?

- **Problem:** SSH istemcinin bu sunucuyu daha önce görmedi. Bu yüzden sunucunun sunduğu **Public Key Fingerprint** (Açık Anahtar Parmak İzi) üzerinden senin onayını bekler.
- **Risk:** Eğer bu uyarıya rağmen bağlanırsan ve aslında araya bir saldırgan girmişse (**Man-in-the-Middle**), tüm trafiğini izleyebilir.
- **Çözüm:** "yes" dediğinde, bu parmak izi senin makinen de `~/.ssh/known_hosts` dosyasına kaydedilir. Gelecekte sunucu farklı bir anahtar sunarsa SSH seni "DİKKAT! Anahtar değişmiş, saldırı olabilir!" diye uyarır.

2. İstemcinin Doğrulanması (Authenticating the Client)

Sunucuya kendimizi tanıtmanın iki yolu vardır:

1. **Parola ile Giriş:** Pratik ama kaba kuvvet (**Brute Force**) saldırılarına karşı zayıftır.
2. **Anahtar Tabanlı Giriş:** Çok daha güvenlidir. Asimetrik şifreleme (Public/Private Key) mantığını kullanır.

ssh-keygen ile Anahtar Üretimi

SSH anahtarı üretmek için `ssh-keygen` aracı kullanılır. Farklı algoritmalar mevcuttur:

- **RSA:** En yaygın olanıdır (CTF'lerde sıkça görülür).
- **ED25519:** Daha kısa anahtar boyutuyla daha yüksek güvenlik ve hız sunar. Günümüzde tavsiye edilen budur.

Komut Kullanımı:

Bash

```
ssh-keygen -t ed25519
```

- **-t:** Algoritma tipini seçer (**rsa**, **ed25519**, **ecdsa** vb.).
- **Passphrase:** Anahtarın çalınsa bile kullanılmasını engellemek için anahtarın üzerine koyduğun yerel bir paroladır. **Dikkat:** Bu parola sunucuya gitmez, sadece senin bilgisayarındaki gizli anahtarı "çözer".

3. SSH Anahtar Dosyaları ve İzinler

Anahtar ürettiğinde iki dosya oluşur:

1. **id_ed25519 (Private Key):** Senin "tapu senedindir". ASLA kimseyle paylaşma.
2. **id_ed25519.pub (Public Key):** Bu senin "kilidindir". Bağlanmak istediğin sunucuya bu dosyayı yüklersin.

Dosya İzinleri (Kritik!)

Linux'ta SSH, gizli anahtar dosyasının izinleri çok genişse (başkaları okuyabiliyorsa) çalışmayı reddeder.

Bash

```
chmod 600 id_ed25519
```

- **600:** Sadece sahibi (sen) okuyabilir ve yazabilir anlamına gelir.

4. Uzak Sunucuya Anahtar Ekleme

Bir sunucuya anahtarınla girmek istiyorsan, **Public Key** içeriğini sunucudaki şu dosyaya eklemelisin: `~/.ssh/authorized_keys`

Pratik Yöntem:

Bash

```
ssh-copy-id -i id_ed25519.pub user@remote_host
```

Bu komut senin açık anahtarını alır ve hedef sunucudaki `authorized_keys` dosyasına otomatik olarak ekler.

5. CTF ve Sızma Testlerinde SSH Anahtarları

Sızma testlerinde SSH anahtarları altın değerindedir:

- **Better Shell (Daha İyi Kabuk):** Bir **Reverse Shell** (ters bağlantı) aldığında bağlantı stabildir değildir (Tab tamamlama çalışmaz, Ctrl+C yapınca bağlantı kopar). Eğer kullanıcının ana dizinine kendi anahtarını ekleyebilirsen, SSH ile bağlanıp tam yetkili ve stabil bir terminal elde edersin.
- **Persistence (Kalıcılık):** Hedef sistemde `authorized_keys` dosyasına kendi anahtarını bırakmak harika bir **Backdoor** (arka kapı) yöntemidir.
- **Private Key Ele Geçirme:** Eğer bir sunucuda bir kullanıcının `id_rsa` (gizli anahtar) dosyasını bulursan, bunu kendi makinene indirip şu komutla şifresiz giriş yapabilirsin:
Bash

```
ssh -i id_rsa user@target_ip
```

(Eğer anahtar parolalıysa, `ssh2john` ve `John the Ripper` kullanarak o parolayı kırmayı deneyebilirsin.)

Dijital İmzalar ve Sertifikalar

Fiziksel dünyada bir belgeyi imzalamak; o belgeyi onayladığınızı, içeriğini kabul ettiğinizi veya bir işlemi yetkilendirdiğinizi kanıtlar. Dijital dünyada ise ıslak imzanın yerini, matematiksel olarak taklit edilmesi imkansız olan **Digital Signatures** (Dijital İmzalar) alır.

1. Dijital İmza Nedir?

Dijital imza, bir mesajın veya dosyanın **Authenticity** (Özgünlük - Kim tarafından gönderildiği) ve **Integrity** (Bütünlük - Değiştirilip değiştirilmediği) özelliklerini doğrulamaya yarar.

Nasıl Çalışır? (Asimetrik Mantık)

Şifrelemenin aksine, dijital imzada anahtarların kullanımı "ters" döner:

- **İmzalama:** Kendi **Private Key**'ini kullanarak dosyayı (veya dosyanın özetini/hash'ini) imzalarsın.
- **Doğrulama:** Herkes senin **Public Key**'ini kullanarak bu imzanın sana ait olup olmadığını kontrol edebilir.

Kritik Fark: Bir dokümanı Public Key ile şifrelemek "Sadece bu kişi okusun" demektir. Bir dokümanı Private Key ile imzalamak ise "Bunu kesinlikle ben gönderdim" demektir.

Teknik Süreç: Hash + Şifreleme

Modern sistemlerde dosyanın tamamı değil, o dosyanın **Hash** değeri (benzersiz özeti) Private Key ile şifrelenir.

1. **Gönderici (Bob):** Dosyanın hash'ini alır, bu hash'i kendi **Private Key**'i ile şifreler (imza oluşur) ve dosyayı imza ile birlikte gönderir.
2. **Alıcı (Alice):**
 - Bob'un **Public Key**'ini kullanarak imzayı çözer (Bob'un hesapladığı hash'e ulaşır).
 - Aldığı dosyanın kendi tarafında hash'ini hesaplar.
 - İki hash birbirine eşitse: Mesaj kesinlikle Bob'dan gelmiştir ve yolda tek bir bit bile değişmemiştir.

2. Sertifikalar: "Sen Gerçekten O musun?"

Dijital imzalar harikadır ancak büyük bir boşluk vardır: Ya bir saldırgan bana kendi Public Key'ini "Bu Google'ın Public Key'idir" diye yutturursa? İşte bu noktada **Certificates** (Sertifikalar) ve **Trust Chain** (Güven Zinciri) devreye girer.

Güven Zinciri (Chain of Trust)

Bir web sitesine (örneğin `tryhackme.com`) girdiğinde tarayıcın sitenin gerçekliğini şu hiyerarşi ile kontrol eder:

1. **Web Sitesi Sertifikası:** Sitenin kimlik bilgilerini ve Public Key'ini içerir.
2. **CA (Certificate Authority - Sertifika Makamı):** Bu sertifikayı imzalayan kurumdur (Örn: DigiCert, Sectigo, Let's Encrypt).
3. **Root CA:** Bilgisayarında, işletim sisteminde veya tarayıcında (Chrome, Firefox) yüklü olarak gelen, dünyaca güvenilen ana makamlardır.

Mantık Zinciri: Tarayıcın der ki: *"Ben Let's Encrypt'e güveniyorum. Let's Encrypt bu sitenin TryHackMe olduğuna dair imza atmış. O halde ben de bu siteye güveniyorum."*

3. HTTPS ve TLS Sertifikaları

Günümüzde her modern web sitesi **HTTPS** kullanmak zorundadır. HTTPS, veriyi **TLS** (Transport Layer Security) sertifikaları ile şifreler.

- **Sertifika Edinme:** Bir web siteniz varsa, yıllık ücret karşılığında CA'lerden sertifika alabilirsiniz.
- **Let's Encrypt:** İnterneti daha güvenli hale getirmek için ücretsiz, otomatik ve açık bir sertifika otoritesi sağlar.
- **Sızma Testi Notu:** Bir sistemde "Self-signed certificate" (Kendi kendine imzalanmış sertifika) uyarısı alıyorsan, bu sertifikanın güvenilir bir makam (CA) tarafından onaylanmadığını, dolayısıyla araya bir saldırganın girmiş olabileceğini (MiTM) gösterir.

Özet Terimler

Terim	Açıklama
Electronic Signature	Bir belgenin üzerine imza resmi yapıştırmak gibidir; teknik bir güvenliği yoktur.
Digital Signature	Matematikseldir; Private Key ile oluşturulur, bütünlüğü ve kimliği kanıtlar.
CA (Certificate Authority)	Dijital kimlikleri (sertifikaları) onaylayan ve imzalayan güvenilir kuruluş.
Let's Encrypt	Ücretsiz TLS sertifikası sağlayan devrimsel bir servis.

PGP ve GPG: E-posta ve Dosya Güvenliğinin Standartları

PGP (Pretty Good Privacy), dosyaları şifrelemek ve dijital olarak imzalamak için geliştirilmiş bir yazılımdır. **GPG** (GnuPG veya GNU Privacy Guard) ise, OpenPGP standartlarını temel alan, tamamen açık kaynaklı bir PGP uygulamasıdır. Siber güvenlik dünyasında özellikle güvenli e-posta iletişimi ve yazılım paketlerinin doğrulanmasında standart olarak kabul edilir.

1. GPG Anahtar Üretimi (GPG Key Generation)

Birine şifreli bir dosya göndermek veya size gönderilen şifreli bir içeriği açmak için bir anahtar çiftine (Public/Private) ihtiyacınız vardır. Linux terminalinde `gpg --full-gen-key` komutu ile bu süreci başlatabiliriz.

Adım Adım Yapılandırma

Aşağıdaki çıktı, bir siber güvenlik uzmanının bilmesi gereken kritik seçimleri içerir:

- Algoritma Seçimi:** Genelde **RSA** veya modern bir seçenek olan **ECC** (Elliptic Curve Cryptography) seçilir. ECC, daha küçük anahtarlarla RSA'ya denk güvenlik sağladığı için (Örn: Curve 25519) tercih edilir.
- Kullanım Amacı:** Anahtarın sadece imzalama (**Sign Only**) mı yoksa hem imzalama hem şifreleme (**Sign and Encrypt**) için mi kullanılacağı belirlenir.
- Geçerlilik Süresi (Expiry Date):** Güvenlik protokolü gereği anahtarlara bir son kullanma tarihi atanması önerilir. CTF'lerde genellikle 0 (sınırsız) seçildiğini görürüz.
- User ID:** Anahtarı kiminle ilişkilendireceğinizi belirten isim ve e-posta bilgisidir.

Bash

```
# Örnek Çıktı Özeti
pub  ed25519 2024-08-29 [SC] # [S]ign ve [C]ertify yetkili anahtar
uid  Strategos <strategos@tryhackme.thm>
sub  cv25519 2024-08-29 [E]  # [E]ncrypt (şifreleme) için alt anahtar
```

2. GPG ile Şifre Çözme ve Anahtar Yönetimi

CTF senaryolarında genellikle karşınıza `.gpg` veya `.pgp` uzantılı şifreli dosyalar ve bir adet "backup" veya "private" key dosyası çıkar. Süreç şu şekilde işler:

Anahtarı İçe Aktarma (Import)

Sisteminize yabancı bir gizli anahtar dosyasını tanıtmak için:

Bash

```
gpg --import backup.key
```

Dosyayı Çözme (Decrypt)

Anahtarı içe aktardıktan sonra, şifreli mesajı şu komutla okuyabilirsiniz:

Bash

```
gpg --decrypt gizli_mesaj.gpg
```

3. Güvenlik ve CTF Teknikleri

Parola Korumalı Anahtarlar

Tıpkı SSH anahtarlarında olduğu gibi, GPG gizli anahtarları da bir **passphrase** (parola) ile korunabilir. Eğer anahtar dosyası elinizdeyse ama parolayı bilmiyorsanız:

- Hash Çıkarma:** `gpg2john private.key > hash.txt` komutuyla anahtarın parolasını bir hash dosyasına dönüştürürsünüz.
- Kaba Kuvvet (Brute Force):** `john --wordlist=rockyou.txt hash.txt` ile bu parolayı kırmaya çalışırsınız.

Altın Kural: Gizli Anahtarın Güvenliği

Açık anahtarınızı (Public Key) herkesle paylaşabilirsiniz; ancak gizli anahtarınızın (**Private Key**) yedeğini güvenli ve şifreli bir yerde saklamanız hayati önem taşır. Gizli anahtar ele geçirilirse, adınıza imzalanmış sahte dokümanlar oluşturulabilir veya tüm geçmiş yazışmalarınız deşifre edilebilir.

Sonuç: Kriptografi ve Kriptanaliz Dengesi

Kriptografi dünyasına giriş yaptığımız bu notları, sistemlerin nasıl korunduğu kadar nasıl saldırıya uğradığını da anlayarak bitirelim. Kriptografi güvenli iletişimi inşa ederken, **Cryptanalysis** (Kriptanaliz) bu sistemleri kırmayı veya atlatmayı hedefler.

Saldırı Türleri ve Temel Kavramlar

Bir siber güvenlik uzmanı olarak, şifrelenmiş bir veriye karşı uygulayabileceğin temel saldırı yöntemleri şunlardır:

1. Brute-Force Attack (Kaba Kuvvet Saldırısı)

Herhangi bir mantık yürütmeden, mümkün olan tüm anahtar veya parola kombinasyonlarını tek tek deneme yöntemidir.

- **Özellik:** Teorik olarak her şifreyi çözer.
- **Zayıflık:** Anahtar uzunluğu arttıkça (örneğin 256-bit AES), bu saldırının tamamlanması için gereken süre evrenin ömründen daha uzun olabilir (**computationally infeasible**).

2. Dictionary Attack (Sözlük Saldırısı)

Tüm kombinasyonları denemek yerine, sadece anlamlı kelimelerin veya daha önce sızdırılmış popüler parolaların (Örn: `rockyou.txt`) bulunduğu listeleri deneme yöntemidir.

- **Mantık:** İnsanlar genellikle karmaşık karakterler yerine hatırlayabilecekleri kelimeleri seçerler.
- **Avantaj:** Brute-force'a göre çok daha hızlı sonuç verir.

Odağımız: Asimetrik Kriptografi Özeti

Bu modülde asimetrik kriptografinin modern internetin omurgasını nasıl oluşturduğunu gördük. İşte aylar sonra hatırlaman gereken özet:

- **RSA:** Çarpanlara ayırma zorluğuna dayanır; hem şifreleme hem imzalama yapar.
- **Diffie-Hellman:** Güvensiz bir hat üzerinden ortak bir gizli anahtar üretmemizi sağlar.
- **SSH Key Pairs:** Parolaya ihtiyaç duymadan, public/private key ikilisiyle güvenli terminal erişimi sağlar.
- **Dijital İmzalar ve Sertifikalar:** Bir mesajın kesinlikle kimden geldiğini (**Authenticity**) ve yolda değişmediğini (**Integrity**) kanıtlayan "Güven Zinciri"ni oluşturur.
- **OpenPGP (GPG):** E-posta ve dosya şifrelemede asimetrik prensipleri kullanan açık kaynak standarttır.

Terminoloji Hatırlatıcı

Terim	Kısa Tanım
Kriptografi	Şifre ve kodlar kullanarak veriyi koruma bilimi.
Kriptanaliz	Anahtarı bilmeden şifreli sistemleri kırma çalışması.
Asimetrik	Birbirinden farklı iki anahtarın (Public/Private) kullanıldığı sistem.
Key Exchange	İki tarafın güvenli bir simetrik anahtar üzerinde anlaşma süreci.